

INF-2200: Computer Architecture and Organization

Mandatory Assignment 2: MIPS Simulator

Jens Christian Valen Leynse

J1e040

j1e040@post.uit.no

Oct 07th, 2020

Introduction

The goal of the assignment is to implement a binary code simulator for a subset of the MIPS architecture. The pipelined datapath and controller will be simulated based on descriptions in the course textbook. The simulator should be able to support 17 instructions, among them are examples such as add, load word, jump, break and so on. The simulator has 4 main tasks; the first is to create an object for each control and datapath element; the second is to connect the elements by specifying input and output data and signals; the third is that for each cycle each element should call the following four commands: readControlSignals(), readInput(), writeOutput() and setControlSingals(), each cycle should also check for break instructions and exit if they are ever encountered; The fourth and last task is to print the content of the registers and the number of the cycles that were executed.

Requirements:

For each cycle:

- For each element, call readControlSignals() and readInput()
- For each element, call writeOutput() and setControlSignals()
- Check if the break instruction has been encountered and exit the loop if so.

Technical Background

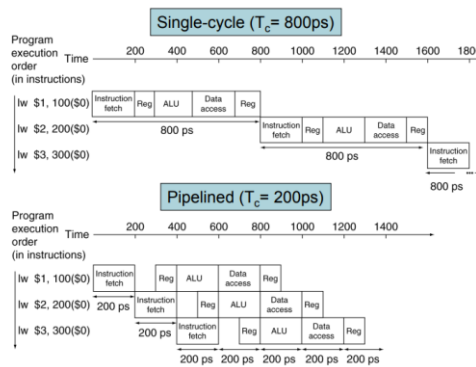
Microprocessor interlocked pipelined stages or MIPS for short is an instruction set architecture (ISA). MIPS can refer to both a hardware implementation of the ISA and the assembly language itself. Although this assignment only handles the software of the assembly language. MIPS can also stand for million instructions per second which is a measurement of execution speed.

Datapaths and controllers

Pipelining in a MIPS processor is a means to improve performance by increasing instruction throughput. The pipelined datapath differs slightly based on what format the instructions have. But it is in general divided into 5 stages:

- IF: Instruction fetch from memory
- ID: Instruction decode and register read
- EX: Execute operation or calculate address
- MEM: Access memory operand
- WB: Write result back to register

Pipelining means to make the sequences of processing the instructions concurrent. Instead of waiting for a datapath/ sequence to finish it executes multiple instructions in parallel. This will result in all the instructions having the same latency excluding any hazards. So, instead of waiting 800ps for the branch to finish, another instruction will execute after the first instruction stage making it only a delay of 200ps.



Since an important part of the MIPS process is handling the registers, they are stored in between stages to hold information from a previous cycle. The control signals derived from the instructions are also stored here.

If it is an R-format instruction:

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

Arithmetic instruction format

The instruction is fetched from the instruction memory, while the pc is incremented. Then the instructions are decoded after which the input registers are read from the register file, the ALU uses the highest bits as it's the funct field to help select the ALU operation and lastly the results from the ALU are written back into the register file using parts of the instructions (rd) to determine the destination register. The R-format instructions consists of arithmetic functions such as; and, or, slt, add, sub, to name a few.

I-Format instruction:

op	rs	rt	address/immediate
----	----	----	-------------------

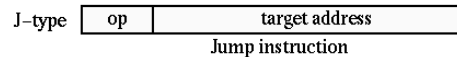
Transfer, branch, immediate.

The instruction does the first three stages the same as the R-format; It fetches from the instruction memory, decodes the instructions as well as reads the registers, and executes the operation. The results from the ALU are then sent as an address to the data memory, were

Commented [JCVL1]: 1 . Summary of the datapath and controllers you are simulating

depending on the function type the data is sent back to the register file using an instruction as an index. The last stage can differ in that store word (sw) does not need to write to the register while the load word (lw) function does, and the branch on equal does not need to access the memory altogether. This difference between the instructions influences the timing of the datapath. Other examples of I-format instructions are bne, addi and lui.

J-Format instruction:



The J-format also as the previous formats performs the first three stages of the pipeline; Instruction fetch from memory, instruction decode as well as register read, and differing a bit on the third step as it calculates the address. The pc has had an adder applied to its output, which is added with the sign-extended lower 16 bits of the offset shifted left by 2 bits, producing the branch target address. The ALU then subtracts its content so that the controller output zero sends a signal that determines which address to write to the next cycles pc. Since the format does not access any data memory or write to the register it is among the fastest in terms of performance. The jump function is the only j-format instruction used.

The setting of the control lines is completely determined by the opcode fields of the instruction. The instruction sent as the only input to the controller uses the highest 6 bits, as that is where the opcode field is. These opcode inputs then determine in turn the controller lines sent as the output. For instance, a load word instruction would tell the controller to send 4 controller output signals.

Hazards

Data hazards are situations where the processor needs to wait for the previous instruction to complete its data accessing. The three different kinds of data hazards are all orders of data accessing excluding read after write, so; write after write, write after read, and read after read. The handling of such hazards can be done with for instance forwarding, which requires extra connections in the datapath adding to its complexity. This allows the CPU to ignore the register file and send a value directly from one instruction to another. This approach however can at times not help at all, as an example when a load instruction is followed by an ALU operation. The destination stage cannot be later in time than the source stage. There is no big cost to forwarding and the only hit the performance takes is that if there's a forwarding issue additional bubbles may be inserted into a number of the branches making the sequences grow by 1 (effectively adding 200ps to a sequence).

Forwarding is at times not possible without another approach, stalling. Stalling usually means to let some instructions in the pipeline proceed, while others wait for data, used by reordering code to avoid using the load results in the next instruction. Stalls also impedes on the progress of the pipeline as 1 stall cycle results in a deviation of 200ps in the clock cycle.

Controller hazards occur when the pipeline cannot fetch the correct instruction. This could be because the process is still working on the instruction decoding stage as the controller is dependent on the branch to fetch its next instruction. Stalling is once again an option, as waiting for the branch to determine its outcome will solve the issue. If the stalling becomes

Commented [JCVL2]: 1. Description of data hazard handling approach including a discussion about the cost, complexity, and performance of the approach
2. Controller hazard handling approach

too long from longer pipelines using predictions is another approach, where stalling is used only when the prediction is wrong.

Design & Implementation

Every element in the datapath firstly uses a connect function which takes four inputs namely; input sources, output value names, control and output signal names. To create a datapath it is essential that the correct number of inputs and outputs are given to each element. Inside the function the number of connections is verified, and local variables are assigned.

The memory file is not present in any processor datapath but is used by both the instruction memory element and the data memory element. As the name suggests it handles all memory file types and performs the following task. It reads the input of the file given, parses through it removing any unwanted comments and adds the memory address and binary code respectively as a key and value to the memory dictionary.

Going from the start of the datapath the instruction memory element uses a given memory file to create its separate instructions. First by comparing the address given by the program counter to an address in the memory dictionary to find the correct binary value. Next this value is converted into a binary string and padded to be the correct length. The binary string is then separated into all the instructions given by the element repeating some over others to allow for usage by different formats.

The highest bit part of the instructions goes to the control element. This part is called the opcode and is used to determine what control lines to send from the element. Some simple if checks are therefore used to check for each relevant opcode input, handling it by changing the default value of the control output lines from a zero to a one for the correct signals.

The register file receives three inputs from the instructions. First the rs field ([25-21]) and the rt field ([20-16]) instructing the first and second register to be read and then a multiplexor output using rt field ([20-16]) and rd field ([15-11]) as a write register inputs. Lastly the write data input is received from a multiplexor after the memory instructions and a register write control input is received as either zero or one from the control element. The register file element handles these inputs by first typecasting all non-control inputs as integers if possible. If the control signal now says to write to the register the third input writes over the fourth in the register dictionary. Then the first two inputs are used as indexes in the register dictionary resulting in the corresponding value being used as output values.

Before the ALU receives the data from the register file the ALU control element must compute what control signals to send it. The three inputs it receives are the function field from the instructions, meaning the lowest bits of the string, and the two ALU operator inputs from the control element. After having assigned default values to all controller outputs if statements check what kind of ALU operation it should pass on. If it is a R-format operation additional if statements are used with the function data to determine the output. All outputs sent are then if necessary, changed in the binary state.

The data memory element takes in both two data inputs and 2 control inputs. The data inputs are the address from the ALU results, and the second register data sent from the register file as a write data input. The controller inputs are either a positive memory read, or memory write. Firstly, the address received by the ALU is used as a key in the memory to get the tuples value, before if statements check for what operations to perform given the control signals. For a positive read the memory value is added to the data memory's read data output, while a positive write means the second input from the register file is used as a memory index for the element to write to.

Commented [JCVL3]: 1 .Description of the simulator implementation, including the functions you have implemented

Some smaller elements that has been implemented are the and gate, the sign extender, and the bitshift element. The and-gate receives two control signals which it checks if both are positive. If they are a positive output is sent as a result, and if not, a zero is sent. The sign extender receives the immediate/ address field in the instructions. It then takes the highest bit and uses it to pad until the length reaches 32 bits. Lastly the bitshifter takes in the largest instruction bit size of any element excluding only the opcode, and after to typecast it twice bitshifts it twice to the left leaving the shifted-out bits as padding.

Testing

All testing of any element uses the same format. A test class is made using “unittest” containing a setup and a correct behavior function. The setup initializes the function and two test element variables in test input and test output. The data sent and received are then defined for each element in one connect test for each one of the three. The data is specific for each element, but it must follow the format of all input types as tuples and all outputs as strings. The test correct behavior function has data manually put in to test the element. Once all control and data inputs has been written the element being tested needs to use the read input or read control signals function to initialize the test. Then the function calls on the elements function definition, either being a write output function or a set control signals function depending on the element. Next the output element needs to read the outputs of the element and some assertion test are done to verify the correct output is sent based on output names.

To specify some elements the ALU test for both “add” and “and” operations and they both receive the same input which is 1 and 0. Meaning the first test output with the “add” opcode should result in a 1 and no zero control signal sent, while the “and” opcode should result in a zero meaning the zero control signal get a positive signal output.

The control element has some of the most extensive testing as there are a lot of variations to be prepared for. It tests for close to one opcode input for each type. By altering the instruction input it receives it checks through all control line outputs to make sure that only the correct ones get a positive result.

Discussion

There are several things to discuss regarding the code. At the point of the deadline there still exists some bugs disabling the code from being able to run a single cycle. The known bugs that could not be solved in time are: An error with the controller inputs to the ALU. The first test should receive three control inputs namely: 0, 1 and 0 resulting in the ALU performing an “add” operation. But through the process of reading the inputs the 1 is not read so the function gives out the wrong output. Another error which is more so an issue regarding time for detailing is the fact that the data memory typecasts its address for an operation typecasting it back afterwards to compare with wanted result. Unfortunately, the tester does not recognize the new hex format resulting in “0x00000064” not being recognized as “0x64”. Because of time issues some controller instructions couldn’t be added in time namely; load upper immediate, break and jump (although jump does get output as a signal). The program also does not check for a break instruction.

The big problem for the code in general is that since a single cycle didn’t manage to make its course through the datapaths a pipeline wasn’t an option to be added (maybe only in theory), it was also given the time of completion not realistic timewise. Thus, adding data hazards and/or controller hazards wasn’t possible.

Commented [JCVL4]: 1 . A detailed description of the test cases you have designed

Commented [JCVL5]: 1 . Summary of known bugs and problems

Conclusion

All the elements for a single cycle MIPS datapath have been added together with the implementations of the functionality and testing functions. At the time of the deadline there still exists bugs/ problems disabling it from being able to be run once to completion. So, adding a pipeline to the datapath and testing for data or control hazards was not implemented. Of the four tasks specified in the introduction all tasks up till the third subtask of the third task has been completed. The third subtask which was to the check for break statement did not get implemented at the time for the deadline.